



MINISTERIO DE EDUCACIÓN DE LA NACIÓN
UNIVERSIDAD NACIONAL DE LA RIOJA
Departamento Académico de Ciencias Exactas, Físicas y Naturales

Circuitos Combinacionales y Secuenciales

ARQUITECTURA DE HARDWARE

Profesor Adjunto: Ing. Juan Esteban Barros

Profesor Ayudante de 1^{ra}: Ing. Matías de la Puente

1. Sistemas Binarios y Hexadecimales

Los computadores se construyen a partir de dispositivos de conmutación que reducen toda la información a ceros y unos, es decir que representan los números con el sistema binario, un sistema que denota todos los números con combinaciones de 2 dígitos. Es decir que el potencial de la computadora se basa en sólo dos estados electrónicos: encendido y apagado. Las características físicas de la computadora permiten que se combinen estos dos estados electrónicos para representar letras, números, colores.

Un estado electrónico de "encendido" o "apagado" se representa por medio de un bit. La presencia o la ausencia de un bit se conoce como un bit encendido o un bit apagado, respectivamente. En el sistema de numeración binario y en el texto escrito, el bit encendido es un 1 y el bit apagado es un 0.

Las computadoras cuentan con software que convierte automáticamente los números decimales en binarios y viceversa. El procesamiento de números binarios de la computadora es totalmente invisible para el usuario humano.

Para que las palabras, frases y párrafos se ajusten a los circuitos exclusivamente binarios de la computadora, se han creado códigos que representan cada letra, dígito y carácter especial como una cadena única de bits. El código más común es el ASCII (American Standard Code for Information Interchange, Código estándar estadounidense para el intercambio de información).

Un grupo de bits puede representar colores, sonidos y casi cualquier otro tipo de información que pueda llegar a procesar un computador.

El sistema numérico binario utiliza como base el 2, que corresponde al número de dígitos que se utilizan para representar cantidades. Estos dígitos son 0 y 1 que se conocen como "bits" abreviadamente. Por tal razón un número binario es una sucesión de bits, posiblemente con un punto binario intercalado.

Al igual que el sistema decimal, el sistema binario es posicional, por lo cual cada dígito tiene un valor relativo a la posición que ocupa en el número. El valor de posición en este sistema se consigue multiplicando el dígito por una potencia de 2. A continuación

Existen infinitas bases de numeración, tantas como posibles valores de B. Sólo unas pocas son de nuestro interés: la base binaria, en la que $B=2$, la base octal, $B=8$, la base decimal, $B=10$ y la base hexadecimal, $B=16$.

	Base 2	Base 8	Base 10	Base hexadecimal
0	0	0	0	0
1	1	1	1	1
2		2	2	2
3		3	3	3
4		4	4	4
5		5	5	5
6		6	6	6
7		7	7	7
8			8	8
9			9	9
10				A
11				B
12				C
13				D
14				E
15				F

Conversión de decimal a binario

Convertir un número decimal al sistema binario es muy sencillo: basta con realizar divisiones sucesivas por 2 y escribir los restos obtenidos en cada división en orden inverso al que han sido obtenidos.

Por ejemplo, para convertir al sistema binario el número 77_{10} haremos una serie de divisiones que arrojarán los restos siguientes:

$$77 : 2 = 38 \text{ Resto: } 1$$

$$38 : 2 = 19 \text{ Resto: } 0$$

$$19 : 2 = 9 \text{ Resto: } 1$$

$$9 : 2 = 4 \text{ Resto: } 1$$

$$4 : 2 = 2 \text{ Resto: } 0$$

$$2 : 2 = 1 \text{ Resto: } 0$$

$$1 : 2 = 0 \text{ Resto: } 1$$

y, tomando los restos en orden inverso obtenemos la cifra binaria: $77_{10} = 1001101_2$

Conversión de binario a decimal

El proceso para convertir un número del sistema binario al decimal es aún más sencillo; basta con desarrollar el número, teniendo en cuenta el valor de cada dígito en su posición, que es el de una potencia de 2, cuyo exponente es 0 en el bit situado más a la derecha, y se incrementa en una unidad según vamos avanzando posiciones hacia la izquierda.

Por ejemplo, para convertir el número binario 1010011_2 a decimal, lo desarrollamos teniendo en cuenta el valor de cada bit:

$$1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 83$$

$$1010011_2 = 83_{10}$$

Conversión de un número decimal a octal

La conversión de un número decimal a octal se hace con la misma técnica que ya hemos utilizado en la conversión a binario, mediante divisiones sucesivas por 8 y colocando los restos obtenidos en orden inverso. Por ejemplo, para escribir en octal el número decimal 122_{10} tendremos que hacer las siguientes divisiones:

$122 : 8 = 15$	Resto: 2
$15 : 8 = 1$	Resto: 7
$1 : 8 = 0$	Resto: 1

Tomando los restos obtenidos en orden inverso tendremos la cifra octal: $122_{10} = 172_8$

Conversión octal a decimal

La conversión de un número octal a decimal es igualmente sencilla, conociendo el peso de cada posición en una cifra octal. Por ejemplo, para convertir el número 237_8 a decimal basta con desarrollar el valor de cada dígito:

$$2 \cdot 8^2 + 3 \cdot 8^1 + 7 \cdot 8^0 = 128 + 24 + 7 = 159_{10}$$

$$237_8 = 159_{10}$$

Conversión octal a decimal

La conversión de un número octal a decimal es igualmente sencilla, conociendo el peso de cada posición en una cifra octal. Por ejemplo, para convertir el número 237_8 a decimal basta con desarrollar el valor de cada dígito:

$$2 \cdot 8^2 + 3 \cdot 8^1 + 7 \cdot 8^0 = 128 + 24 + 7 = 159_{10}$$

$$237_8 = 159_{10}$$

Conversión de un número decimal a hexadecimal

Ensayemos, utilizando la técnica habitual de divisiones sucesivas, la conversión de un número decimal a hexadecimal. Por ejemplo, para convertir a hexadecimal del número 1735_{10} será necesario hacer las siguientes divisiones:

$1735 : 16 = 108$	Resto: 7
$108 : 16 = 6$	Resto: C es decir, 12_{10}
$6 : 16 = 0$	Resto: 6

De ahí que, tomando los restos en orden inverso, resolvemos el número en hexadecimal:

$$1735_{10} = 6C7_{16}$$

Conversión de números binarios a octales y viceversa

Observa la tabla siguiente, con los siete primeros números expresados en los sistemas decimal, binario y octal:

DECIMAL	BINARIO	OCTAL
0	000	0
1	001	1
2	010	2
3	011	3
4	100	4
5	101	5
6	110	6
7	111	7

Cada dígito de un número octal se representa con tres dígitos en el sistema binario. Por tanto, el modo de convertir un número entre estos sistemas de numeración equivale a "expandir" cada dígito octal a tres dígitos binarios, o en "contraer" grupos de tres caracteres binarios a su correspondiente dígito octal.

Por ejemplo, para convertir el número binario 101001011_2 a octal tomaremos grupos de tres bits y los sustituiremos por su equivalente octal:

$$101_2 = 5_8$$

$$001_2 = 1_8$$

$$011_2 = 3_8$$

$$\text{y, de ese modo: } 101001011_2 = 513_8$$

La conversión de números octales a binarios se hace, siguiendo el mismo método, reemplazando cada dígito octal por los tres bits equivalentes. Por ejemplo, para convertir el número octal 750_8 a binario, tomaremos el equivalente binario de cada uno de sus dígitos:

$$7_8 = 111_2$$

$$5_8 = 101_2$$

$$0_8 = 000_2$$

$$\text{y, por tanto: } 750_8 = 111101000_2$$

Conversión de números binarios a hexadecimales y viceversa

Del mismo modo que hallamos la correspondencia entre números octales y binarios, podemos establecer una equivalencia directa entre cada dígito hexadecimal y cuatro dígitos binarios, como se ve en la siguiente tabla:

DECIMAL	BINARIO	HEXADECIMAL
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

La conversión entre números hexadecimales y binarios se realiza "expandiendo" o "contrayendo" cada dígito hexadecimal a cuatro dígitos binarios. Por ejemplo, para expresar en hexadecimal el número binario 101001110011_2 bastará con tomar grupos de cuatro bits, empezando por la derecha, y reemplazarlos por su equivalente hexadecimal:

$$\begin{aligned} 1010_2 &= A_{16} \\ 0111_2 &= 7_{16} \\ 0011_2 &= 3_{16} \\ \text{y, por tanto: } 101001110011_2 &= A73_{16} \end{aligned}$$

En caso de que los dígitos binarios no formen grupos completos de cuatro dígitos, se deben añadir ceros a la izquierda hasta completar el último grupo. Por ejemplo: $101110_2 = 00101110_2 = 2E_{16}$

La conversión de números hexadecimales a binarios se hace del mismo modo, reemplazando cada dígito hexadecimal por los cuatro bits equivalentes de la tabla. Para convertir a binario, por ejemplo, el número hexadecimal 1F616 hallaremos en la tabla las siguientes equivalencias:

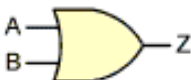
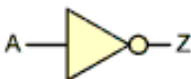
$$\begin{aligned} 1_{16} &= 0001_2 \\ F_{16} &= 1111_2 \\ 6_{16} &= 0110_2 \\ \text{y, por tanto: } 1F6_{16} &= 000111110110_2 \end{aligned}$$

2. Álgebra de Boole y Compuertas Lógicas

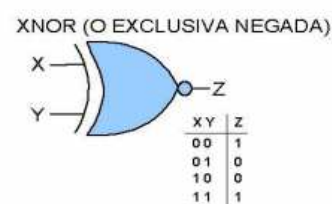
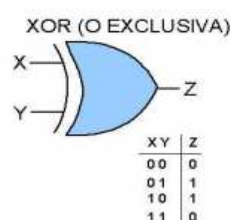
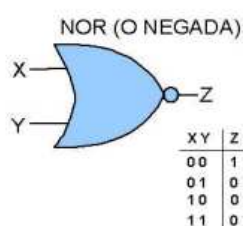
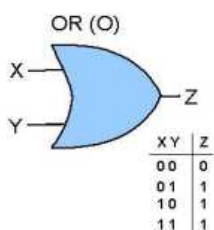
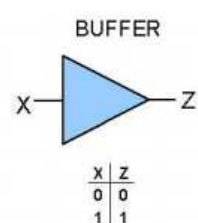
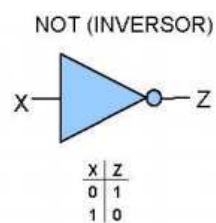
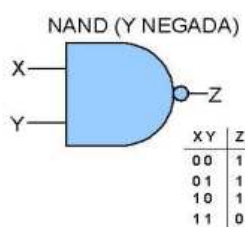
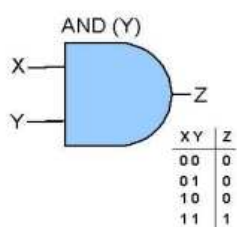
Debido a que los computadores trabajan con información binaria, la herramienta matemática adecuada para el análisis y diseño de su funcionamiento es el Álgebra de Boole en su forma bivalente, aunque fue desarrollada inicialmente para el estudio de la lógica. Ha sido a partir de 1938, fecha en que C.E. Shannon publicó su obra *Análisis simbólico de circuitos con relés*, estableciendo los primeros conceptos de la actual teoría de la conmutación, cuando se ha producido un aumento considerable en el número de trabajos de aplicación del Álgebra de Boole a los computadores digitales. Hoy en día, esta herramienta resulta fundamental para el desarrollo de los computadores ya que, con su ayuda, el análisis y síntesis de combinaciones complejas de circuitos lógicos puede realizarse con rapidez y eficacia.

Para que el Álgebra de Boole se torne realmente útil de cara a la electrónica y la computación, ésta debe plantearse como un álgebra bivalente. No hay acuerdo acerca de si tal Álgebra "nació" bivalente, o el ser bivalente es una restricción añadida para facilitar su aplicación. A este respecto, Kneale y Bochénski mantienen opiniones contrapuestas⁶.

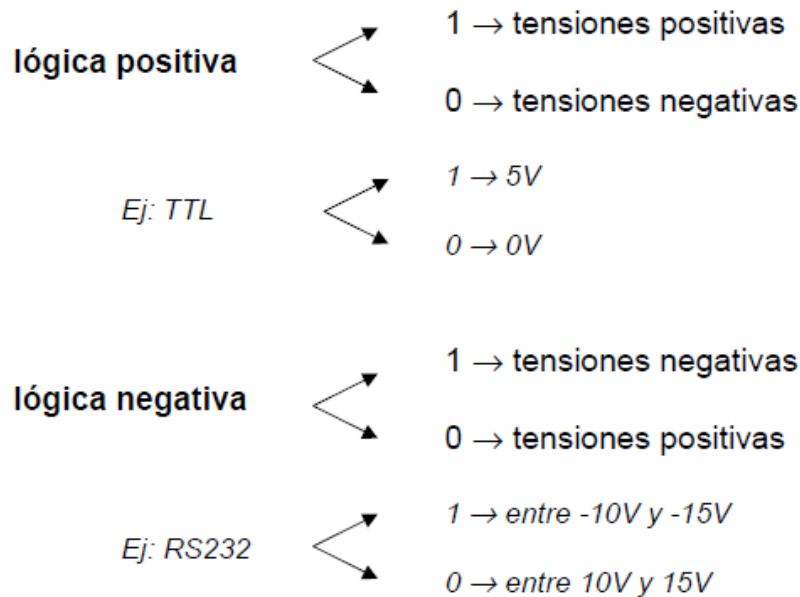
En cualquier caso, este álgebra bivalente aplicada tiene las mismas tablas de verdad del CP expuestas anteriormente, cambiándoles sólo la nomenclatura: donde decíamos "disyunción" ($\vee$), ahora decimos OR; donde decíamos "conjunción" ($\wedge$), decimos AND; donde decíamos "negación" ($\neg$), ahora decimos INVERSOR o NOT. Veamos de nuevo la tabla con el vocabulario renovado:

CONECTOR/COMPUERTA, ENTRADA(S), SALIDA CONNECTOR/GATE, INPUT(S), OUTPUT	NOMBRE NAME	TABLA DE VERDAD TRUTH TABLE															
	AMORTIGUADOR BUFFER	<table><tr><th>A</th><th>Z</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	A	Z	0	0	1	1									
A	Z																
0	0																
1	1																
	Y AND	<table><tr><th>A</th><th>B</th><th>Z</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Z	0	0	0	1	0	0	0	1	0	1	1	1
A	B	Z															
0	0	0															
1	0	0															
0	1	0															
1	1	1															
	O (O, en sentido inclusivo) OR	<table><tr><th>A</th><th>B</th><th>Z</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Z	0	0	0	1	0	1	0	1	1	1	1	1
A	B	Z															
0	0	0															
1	0	1															
0	1	1															
1	1	1															
	OE (O, en sentido exclusivo) XOR (EXCLUSIVE-OR)	<table><tr><th>A</th><th>B</th><th>Z</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Z	0	0	0	1	0	1	0	1	1	1	1	0
A	B	Z															
0	0	0															
1	0	1															
0	1	1															
1	1	0															
	N, NEG o INVERSOR NOT or INVERTER	<table><tr><th>A</th><th>Z</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	Z	0	1	1	0									
A	Z																
0	1																
1	0																
	NY (N Y) NAND (NOT AND)	<table><tr><th>A</th><th>B</th><th>Z</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Z	0	0	1	1	0	1	0	1	1	1	1	0
A	B	Z															
0	0	1															
1	0	1															
0	1	1															
1	1	0															
	NO (N O) NOR (NOT OR)	<table><tr><th>A</th><th>B</th><th>Z</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Z	0	0	1	1	0	0	0	1	0	1	1	0
A	B	Z															
0	0	1															
1	0	0															
0	1	0															
1	1	0															
	NOE (N OE) NXOR (NOT EXCLUSIVE-OR)	<table><tr><th>A</th><th>B</th><th>Z</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Z	0	0	1	1	0	0	0	1	0	1	1	1
A	B	Z															
0	0	1															
1	0	0															
0	1	0															
1	1	1															

FUNCIÓN	SÍMBOLO AMERICANO	SÍMBOLO EUROPEO	ECUAC. LÓGICA	ESQUEMA ELÉCTRICO	TABLA DE ESTADOS	COMENTARIOS															
Puerta NO INVERT (no inversor) o IGUALDAD			$S = A$		<table><tr><th>E</th><th>S</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	E	S	0	0	1	1	Utilizada para el acoplamiento de circuitos sin inversión de la señal de entrada									
E	S																				
0	0																				
1	1																				
Puerta INVERT (inversor) o NOT			$S = \bar{A}$		<table><tr><th>E</th><th>S</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	E	S	0	1	1	0	Invierte el estado de la señal aplicada a su entrada									
E	S																				
0	1																				
1	0																				
Puerta OR (suma lógica)			$S = A + B$		<table><tr><th>B</th><th>A</th><th>S</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	B	A	S	0	0	0	0	1	1	1	0	1	1	1	1	La salida es "0" cuando todas las entradas son "0"
B	A	S																			
0	0	0																			
0	1	1																			
1	0	1																			
1	1	1																			
Puerta AND (producto lógico)			$S = A * B$		<table><tr><th>B</th><th>A</th><th>S</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	B	A	S	0	0	0	0	1	0	1	0	0	1	1	1	La salida es "1" cuando todas las entradas son "1"
B	A	S																			
0	0	0																			
0	1	0																			
1	0	0																			
1	1	1																			
Puerta NOR (suma negada)			$S = \overline{A + B}$		<table><tr><th>B</th><th>A</th><th>S</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	B	A	S	0	0	1	0	1	0	1	0	0	1	1	0	La salida es "1" solo cuando todas las entradas son "0"
B	A	S																			
0	0	1																			
0	1	0																			
1	0	0																			
1	1	0																			
Puerta NAND (producto lógico negado)			$S = \overline{A * B}$		<table><tr><th>B</th><th>A</th><th>S</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	B	A	S	0	0	1	0	1	1	1	0	1	1	1	0	La salida es "0" cuando todas las entradas son "1"
B	A	S																			
0	0	1																			
0	1	1																			
1	0	1																			
1	1	0																			
Puerta EXOR (OR exclusiva)			$S = A \oplus B = A\bar{B} + \bar{A}B$		<table><tr><th>B</th><th>A</th><th>S</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	B	A	S	0	0	0	0	1	1	1	0	1	1	1	0	La salida es "0" cuando todas las entradas son iguales
B	A	S																			
0	0	0																			
0	1	1																			
1	0	1																			
1	1	0																			
Puerta EXNOR (NOR exclusiva)			$S = \overline{A \oplus B} = A\bar{B} + \bar{A}B$		<table><tr><th>B</th><th>A</th><th>S</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	B	A	S	0	0	1	0	1	0	1	0	0	1	1	1	La salida es "1" cuando todas las entradas son iguales
B	A	S																			
0	0	1																			
0	1	0																			
1	0	0																			
1	1	1																			



Representación de variables lógicas



- En 1815 *George Boole* propuso una herramienta matemática llamada **Álgebra de Boole**.
- Luego en 1938 *Claude Shannon* propuso que con esta álgebra es posible modelar los llamados **Sistemas Digitales**.
- El **Álgebra de Boole** es un sistema matemático que utiliza variables y operadores lógicos. Las variables pueden valer 0 ó 1. Y las operaciones básicas son **OR(+)** y **AND(·)**.
- Luego se definen las expresiones de conmutación como un número finito de variables y constantes, relacionadas mediante los operadores (**AND** y **OR**).
- En la ausencia de paréntesis, se utilizan las mismas reglas de precedencia, que tienen los operadores suma (**OR**) y multiplicación (**AND**) en el álgebra normal.

Leyes

En el álgebra de Boole se cumplen las siguientes Leyes:

- 1) **Conmutatividad:**

$$X + Y = Y + X$$

$$X \cdot Y = Y \cdot X$$

- 2) **Asociatividad:**

$$X + (Y + Z) = (X + Y) + Z$$

$$X \cdot (Y \cdot Z) = (X \cdot Y) \cdot Z$$

- 3) **Distributividad:**

$$X + (Y \cdot Z) = (X + Y) \cdot (X + Z)$$

$$X \cdot (Y + Z) = (X \cdot Y) + (X \cdot Z)$$

- 4) **Elementos Neutros (Identidad):**

$$X + 0 = X$$

$$X \cdot 1 = X$$

- 5) **Complemento:**

$$X + \overline{X} = 1$$

$$X \cdot \overline{X} = 0$$

- 6) **Dominación:**

$$X + 1 = 1 \quad X \cdot 0 = 0$$

Demostración:

$$X + 1 = (X + 1) \cdot 1 = (X + 1) \cdot (X + \overline{X})$$

$$(X + 1) \cdot (X + \overline{X}) = X + (1 \cdot \overline{X}) = 1$$

- 7) Idempotencia:

$$X + X = X$$

$$X \cdot X = X$$

- 8) Doble complemento:

$$\overline{\overline{X}} = X$$

- 9) Absorción:

$$X + X \cdot Y = X$$

$$X \cdot (Y + X) = X$$

Demostración:

$$X + X \cdot Y = (X \cdot 1) + (X \cdot Y) = X \cdot (1 + Y) = X$$

- 10) DeMorgan:

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

Teoremas

Luego se establecen los siguientes Teoremas:

- Teorema de la Simplificación

$$A + \overline{A} \cdot B = A + B$$

$$A \cdot (\overline{A} + B) = A \cdot B$$

Demostración: $\rightarrow$

$$A \cdot \overline{A} = 0$$

$$A \cdot \overline{A} + B = B$$

$$(A + B) \cdot (\overline{A} + B) = B$$

$$A \cdot (A + B) \cdot (\overline{A} + B) = A \cdot B$$

$$A \cdot (\overline{A} + B) = A \cdot B$$

- **Teorema del complemento único**

Suponemos 2 complementos para A (A_1 y A_2)

$$A + A_1 = 1 \quad A + A_2 = 1$$

$$A \cdot A_1 = 0 \quad A \cdot A_2 = 0$$

Luego,

$$A_1 = A_1 \cdot 1 = A_1 \cdot (A + A_2) = A_1 \cdot A + A_1 \cdot A_2$$

$$A_1 = 0 + A_2 \cdot A_1$$

$$A_1 = A \cdot A_2 + A_1 \cdot A_2 = (A + A_1) \cdot A_2$$

$$A_1 = 1 \cdot A_2 = A_2$$

Algunas definiciones:

- **Expresión Dual:** Esta expresión se obtiene, intercambiando las operaciones **AND** por **OR** (y vice versa), e intercambiando las constantes 0 por 1 y 1 por 0 en la expresión de conmutación.

Por ejemplo, para la expresión de conmutación:

$$(A \cdot B) + (C \cdot D) + 0$$

La Expresión Dual es:

$$(A + B) \cdot (C + D) \cdot 1$$

- Las funciones de conmutación se pueden expresar: de **Forma Algebraica**, mediante una **Tabla de Verdad** o en **Forma Canónica**.
- La manera más didáctica de representar una función de conmutación es mediante una **Tabla de Verdad**, ya que en ella se muestran los valores de salida para cada combinación de valor de entrada.
- Las Tablas de Verdad permiten modelar los Sistemas Combinacionales.

- ▶ No existe una forma mecánica y fácil para reducir una función, hay que practicar.
- ▶ De esto se deduce que no hay una única forma de escribir una función lógica, surge la necesidad de las **formas canónicas**.
- ▶ La idea es, **dada una tabla de verdad escribir una expresión booleana que la represente**.
- ▶ Las dos técnicas que vamos a ver son Suma de Productos y Producto de Sumas.
- ▶ No necesariamente vamos a obtener la expresión "Óptima" (o sea la que use menos operadores).

Suma de productos

- ▶ Por cada valor de la función que sea **1** escribimos un término utilizando todas las variables unidas por operadores AND, de forma tal que el término también valga **1**. Luego combinamos todo con operadores OR.
- ▶ Probemos con un ejemplo sencillo:

A	B	F(A,B)
0	0	0
0	1	1
1	0	1
1	1	0

$$F(A, B) = \overline{A}B + A\overline{B}$$

(Es el operador OR-Exclusivo o XOR)

Producto de sumas

- Por cada valor de la función que sea 0 escribimos un término utilizando todas las variables unidas por operadores OR, de forma tal que el término también valga 0. Luego combinamos todo con operadores AND.
- Usando el ejemplo anterior:

A	B	F(A,B)
0	0	0
0	1	1
1	0	1
1	1	0

$$F(A, B) = (A + B)(\bar{A} + \bar{B})$$

Ejercicio 1

Demostrar si la siguiente igualdad entre expresiones booleanas es verdadera o falsa:

$$(X + \bar{Y}) = \overline{(\bar{X}.Y)}.Z + X.\bar{Z} + \overline{(Y + Z)}$$

Una solución:

$$\overline{(\bar{X}.Y)}.Z + X.\bar{Z} + \overline{(Y + Z)} \leftarrow \text{De Morgan}$$

$$\overline{(\bar{X}.Y)}.Z + X.\bar{Z} + \bar{Y}.\bar{Z} \leftarrow \text{Distributiva}$$

$$(\bar{X}.Y).Z + (X + \bar{Y}).\bar{Z} \leftarrow \text{De Morgan}$$

$$(X + \bar{Y}).Z + (X + \bar{Y}).\bar{Z} \leftarrow \text{Distributiva}$$

$$(X + \bar{Y}).(Z + \bar{Z}) \leftarrow \text{Inverso}$$

$$(X + \bar{Y}).1 \leftarrow \text{Identidad}$$

$$X + \bar{Y} \leftarrow \text{Listo!, probamos que es igual}$$

Ejercicio 2

Dada la siguiente tabla de verdad:

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

- Escribir una expresión booleana que la represente.
- Implementarla utilizando a lo sumo una compuerta AND, una compuerta OR y una compuerta NOT.
- Expresamos como una suma de productos:

$$(\overline{A}.\overline{B}.C) + (\overline{A}.B.C) + (A.B.C)$$

- Como nos restringen la cantidad de compuertas tenemos que simplificar.

$$(\overline{A}.\overline{B}.C) + (\overline{A}.B.C) + (A.B.C) \rightarrow \text{Distributiva}$$

$$((\overline{A}.\overline{B}) + (\overline{A}.B) + (A.B)).C \rightarrow \text{Distributiva}$$

$$((\overline{A}.\overline{B}) + (\overline{A} + A).B).C \rightarrow \text{Inverso}$$

$$((\overline{A}.\overline{B}) + 1.B).C \rightarrow \text{Identidad}$$

$$((\overline{A}.\overline{B}) + B).C \rightarrow \text{Distributiva}$$

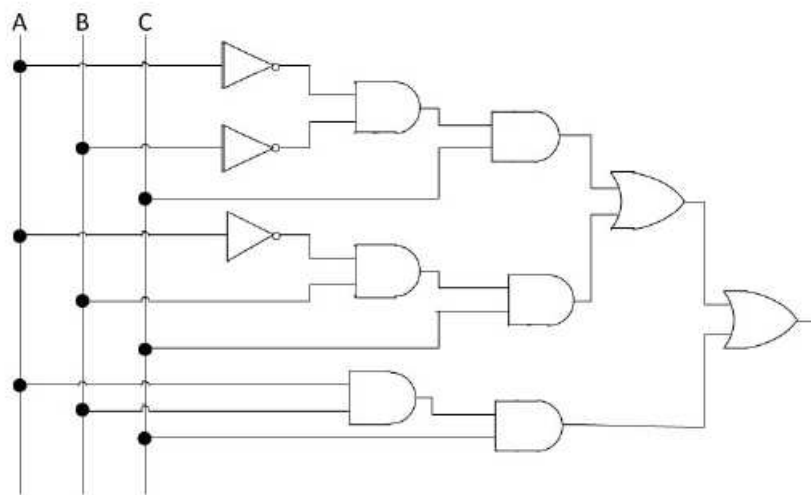
$$((\overline{A} + B).(\overline{B} + B)).C \rightarrow \text{Inverso}$$

$$((\overline{A} + B), 1).C \rightarrow \text{Identidad}$$

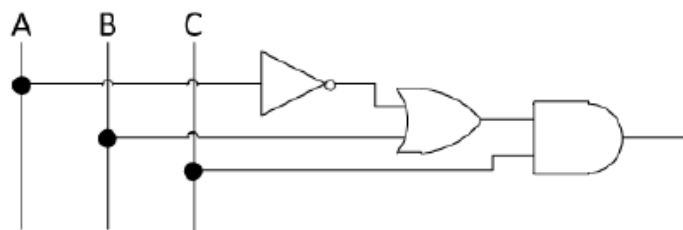
$$(\overline{A} + B).C \rightarrow \text{Bingo!}$$

¿Para que nos sirvió esto?

Implementación de la primer expresión.



Implementación de la segunda expresión.



Cosas que tendríamos que haber entendido y tips

- ▶ Operadores y expresiones booleanas, reducciones utilizando propiedades.
- ▶ Dada una tabla de verdad escribir una expresión booleana que la represente.
- ▶ Implementar las expresiones utilizando compuertas lógicas.

RECOMENDACIONES Sean cuidadosos cuando dibujan circuitos:

- ▶ Que quede claro cuando un cable esta conectado a otro y cuando lo saltea (pongan un circulito en la unión o una curva cuando no quieren que lo toque).